1.01 Number systems

Denary numbers

As a child we first encounter the numbers that we use in everyday life when we are first learning to count. Specifically, we learn to count using 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. This gives us ten different symbols to represent each individual digit. This is therefore a base-10 number system. Numbers in this system are called **denary numbers** or, more traditionally, decimal numbers.



KEY TERM

Denary numbers: which are also known as decimal numbers are written using one of the symbols 0,1,2,3,4,5,6,7,8,9 for each denary digit

When a number is written down the value that it represents is defined by the place values of the digits in the number. This can be illustrated by considering the denary number 346 which is interpreted as shown in Table 1.01.

Place value	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
Digit	3	4	6
Product of digit and place value	300	40	6

Table 1.01 Use of place values in the representation of a denary number

You can see that starting from the right-hand end of the number (which holds the least significant digit), the place value increases by the power of the base number.

Binary numbers

The binary number system is base-2. Each binary digit is written with either of the symbols 0 and 1. A binary digit is referred to as a **bit**.



KEY TERM

Bit: a digit in the binary number system written using either of the symbols 0 and 1

As with a denary number, the value of a binary number is defined by place values. For example, see Table 1.02 for the binary number 101110.

Place value	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Digit	1	0	1	1	1	0
Product of digit and place value	32		8	4	2	0

Table 1.02 Use of place values in the representation of a binary number

By adding up the values in the bottom row you can see that the binary number 101110 has a value which is equivalent to the denary number 46.

You must be able to use the binary number system in order to understand computer systems. This is because inside computer systems there is no attempt made to represent ten

different digits individually. Instead, all computer technology is engineered with components that represent or recognise only two states: 'on' and 'off'. To match this, all software used by the hardware uses binary codes which consist of bits. The binary code may represent a binary number but this does not have to be the case.

Binary codes are most often based on the use of one or more groups of eight bits. A group of eight bits is called a **byte**.

KEY TERM

Byte: a group of eight bits treated as a single unit

Hexadecimal numbers

These are base-16 numbers where each hexadecimal digit is represented by one of the following symbols: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F. The symbols A through to F represent the denary values 10 through to 15. The value of a number is defined by place values. For example, see Table 1.03 for the hexadecimal number 2A6.

Place value	$16^2 = 256$	$16^1 = 16$	$16^0 = 1$
Digit	2	A	6
Product of digit and place value	512	160	6

Table 1.03 Use of place values in the representation of a hexadecimal number

Adding up the values in the bottom row shows that the equivalent denary number is 678.

In order to explain why hexadecimal numbers are used we need first to define the **nibble** as a group of four bits.

KEY TERM

Nibble: a group of four bits

A nibble can be represented by one hexadecimal digit. This means that each byte of binary code can be written as two hexadecimal digits. Two examples are shown in Table 1.04 together with their denary equivalent.

Binary	Hexadecimal	Denary
00001010	0A	10
11111111	FF	255

Table 1.04 Examples of a byte represented by two hexadecimal digits

Note here that if you were converting the binary number 1010 to a hexadecimal number as an exercise on a piece of paper you would not bother with including leading zeros. However, a binary code must not include blanks; all positions in the byte must have either a 0 or a 1. This is followed through in the hexadecimal representation.

One example when you will see hexadecimal representations of bytes is when an error has occurred during the execution of a program. A memory dump could be provided which has a hexadecimal representation of the content of some chosen part of the memory. Another use is when the bytes contain binary numbers in the charts that define character codes. This is discussed later in this chapter.

In the character code charts and in other online sources you may see references to octal numbers which are base-8. You can ignore these.

Converting between binary and denary numbers

One method for converting a binary number to a denary number is to add up the place values for every digit that has a value 1. This was illustrated in Table 1.02.

An alternative method is shown in Worked Example 1.01.

WORKED EXAMPLE 1.01

To carry out the conversion you start at the most significant bit and successively multiply by two and add the result to the next digit. The following shows the method being used to convert the binary number 11001 to the denary number 25:

	1	×	2	=	2
add 2 to 1, then	2	×	3	=	6
add 6 to 0, then	2	×	6	=	12
add 12 to 0, then	2	×	12	=	24

add 24 to 1 to give 25.

To convert a denary number to binary begin by identifying the largest power of 2 that has a value less than the denary number. You can then write down the binary representation of this power of 2 value. This will be a 1 followed by the appropriate number of zeros.

Now subtract the power of two value from the denary number. Then identify the largest power of 2 value that is less than the remainder from the subtraction. You can now replace a zero in the binary representation with a 1 for this new power of 2 position.

Repeat this process until you have accounted for the full denary number.

For example, for the denary number 78 the largest power of two value less than this is 64 so you can start by writing down 1000000. The remainder after subtracting 64 from 78 is 14. The largest power of two value less than this is 8 so the replacement of a zero by 1 gives 1001000. Repeating the process finds values of 4 then 2 so the final answer is 1001110.

An alternative approach is shown in Worked Example 1.02.

WORKED EXAMPLE 1.02

A useful way to convert a denary value to its binary equivalent is the procedure of successive division by two with the remainder written down at each stage. The converted number is then given as the set of remainders in reverse order.

This can be illustrated by the conversion of denary 246 to binary:

246	÷	2	→	123	with remainder 0
123	÷	2	→	61	with remainder 1
61	÷	2	→	30	with remainder 1
30	÷	2	→	15	with remainder 0
15	÷	2	→	7	with remainder 1
7	÷	2	→	3	with remainder 1
3	÷	2	→	1	with remainder 1
1	÷	2	→	0	with remainder 1

Thus, the binary equivalent of denary 246 is 11110110.

TIP

To check that an answer with eight bits is sensible, remember that the maximum denary value possible in seven bits is $2^7 - 1$ which is 127 whereas eight bits can hold values up to $2^8 - 1$ which is 255.

Conversions for hexadecimal numbers

It is possible to convert a hexadecimal number to denary by using the method shown in Table 1.03. However, if there are more than a few digits, the numbers involved in the conversion become very large. Instead, the sensible approach is to first convert the hexadecimal number to a binary number which can then be converted to denary.

To convert a hexadecimal number to binary, each digit is treated separately and converted into a 4-bit binary equivalent, remembering that F converts to 1111, E converts to 1110 and so on.

To convert a binary number to hexadecimal you start with the four least significant bits and convert them to one hexadecimal digit. You then proceed upwards towards the most significant bit, successively taking groupings of four bits and converting each grouping to the corresponding hexadecimal digit.

TASK 1.01

Convert each of the denary numbers 96, 215 and 374 into hexadecimal numbers.
Convert each of the hexadecimal numbers B4, FF and 3A2C to denary numbers.

Question 1.01

Does a computer ever use hexadecimal numbers?

1.02 Numbers and quantities

There are several different types of numbers within the denary system. Examples of these are provided in Table 1.05.

Type of number	Examples	Comments
Integer	3 or 47	A whole number used for counting
Signed integer	−3 or 47	The positive number has an implied + sign
Fraction	2/3 or 52/17	Rarely used in computer science
A number with a whole number part and a fractional number part	−37.85 or 2.83	The positive number has an implied + sign
A number expressed in exponential notation	-3.6×10^8 or 4.2×10^{-9}	The value can be positive or negative and the exponent can be positive or negative

Table 1.05 Different ways to express a value using the denary number system

We will focus on how large values are represented. If we have a quantity that includes units of measurement, it can be written in three different ways. For example, a distance could be written in any one of these three ways:

- 23 567 m
- 23.567×10^3 m
- 23.567 km